

HQ-Employee

01 / 08

The Governed AI Business Employee

Real-time 24 kHz speech discovery and client coordination bounded by deterministic commercial policy.

- ❖ Built on AssemblyAI's managed Voice Agent API (<wss://agents.assemblyai.com/v1/ws>)
- ❖ Native 24 kHz full-duplex speech with immediate barge-in interruption
- ❖ Deterministic tri-state Policy Engine (ALLOW / REQUIRE_APPROVAL / BLOCK)
- ❖ Zero secret leakage via single-use ephemeral HMAC voice tickets

The Enterprise Voice AI Challenge

02 / 08

Why Traditional Conversational Bots Fail in Production

Unconstrained reasoning models cannot be trusted with commercial and legal authority.

- ❖ Hallucinated Pricing: LLMs invent discounts and project estimates without business boundaries
- ❖ Lack of Authority: No mathematical distinction between browsing info and signing contracts
- ❖ Turn-Taking Protocol Desync: Naive tool execution causes speech stutter, audio overlap, and desync
- ❖ Credential Leakage: Exposing raw API keys to browser clients violates enterprise security

Governed Architecture & Zero-Trust Security

03 / 08

Single-Origin Delivery with Ephemeral Ticket Gating

The reasoning model is treated as an untrusted worker governed by deterministic gateway logic.

- ❖ Browser Console: AudioWorklet captures 24 kHz PCM16 mono audio with zero external dependencies
- ❖ Single-Use HMAC Tickets: Client requests 60s ticket (GET /api/voice/ticket); raw API keys stay server-side
- ❖ Strict Gateway Controls: Reused tickets (4003), expired tickets, invalid origins, and duplicate IPs (4029) rejected
- ❖ Append-Only Audit Log: Every action, decision, reason, timestamp, and actor ID recorded with credential sanitization

AssemblyAI Voice Agent API Integration

04 / 08

Native 24 kHz Full-Duplex Speech & Strict Handshake

*Direct WebSocket integration to `wss://agents.assemblyai.com/v1/ws` with *bidirectional* audio.*

- ❖ Deterministic Handshake: `session.update` sent as first upstream message; `session.ready` confirms `session_id`
- ❖ Zero Audio Dropping: Microphone frames buffered before `session.ready` and drained immediately upon ready
- ❖ True Barge-In Interruption: `voice.speech_started` halts agent playback and flushes buffers instantly
- ❖ Low-Latency Neural Speech: Real-time turn detection and high-fidelity speech synthesis

Docs-Conformant Tool Coordination (BLK-012)

05 / 08

Official Turn-Taking Protocol for Client-Side Tools

Tool execution synchronized to the exact AssemblyAI event specification.

- ❖ `tool.call` Arrives: Execution runs; result is buffered during active agent speech (`reply.started`)
- ❖ `reply.done` Arrives: Coordinator drains accumulated results and sends `tool.result` to upstream socket
- ❖ Interruption Protection: If speech is interrupted before `reply.done`, pending results are cleanly discarded
- ❖ 13 Registered Business Tools: Strict JSON schemas governing discovery, budget, timeline, and calendar

Deterministic Policy Engine

06 / 08

Fail-Closed Tri-State Commercial Boundaries

The AI reasoning engine never decides its own business or legal authority.

- ❖ ALLOW: Permitted routine operations (schedule_meeting, record_budget, record_timeline)
- ❖ REQUIRE_APPROVAL: Custom discount requests (e.g. 15% discount) escalate to human Commercial Director
- ❖ BLOCK: Legal agreements (sign_contract), wire transfers, and credential access strictly blocked
- ❖ Immutable Audit Trail: 100% of policy evaluations logged with decision provenance and timestamp

Empirical Production Verification

07 / 08

10 / 10 Live Integration Checks Passed

Grounded in real empirical outputs from docs/PRE_SUBMISSION_VERIFICATION.md.

- ❖ Health & Gateways: GET /health/live and /health/ready return 200 OK; single-use ticket lifecycle verified
- ❖ Audio & Protocol: 24 kHz WAV streaming confirmed with user transcript, agent speech, and reply.done
- ❖ Governance & Lifecycle: ALLOW/REQUIRE_APPROVAL/BLOCK verified; clean socket teardown on disconnect
- ❖ Test Suite & Security: 181 / 181 automated tests passing across 24 test suites; zero secret leaks

Deployment & Honest Disclosures

08 / 08

Single-Origin Production Reality

Full transparency on runtime architecture, persistence, and external modalities.

- ❖ Google Cloud Run: Single-instance container (min=max=1, session-affinity, no-cpu-throttling)
- ❖ Persistence Model: Authoritative state in Node.js process memory; resets on container reboot (PERSISTENCE_STATUS.md)
- ❖ Database Schemas: Structured PostgreSQL migrations (001–006) for non-blocking write-through
- ❖ External Modalities: Google Calendar with simulated fallback; telephony SIP dispatch simulated; Android UI prototype